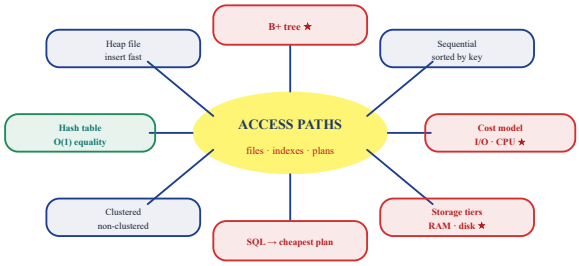
File Organization, Indexing & Query Optimization

Heap · Sequential · Hashed · Indexed · B+ tree · hash · cost-based optimizer

Eleven sheets: the physical layout of a file, where the bytes live, how an index finds a row, the two access structures, how the optimizer picks a plan, and a full worked sum-up.



↑ the whole topic: red = high-yield, green = the other access structure

Syllabus coverage

The syllabus names this lesson clause by clause: "**File organization and data storage; Indexing: primary and secondary indexes, B+ trees, hash tables; BCS Query optimization;" All three clauses are taken in full here.** db1 designed the relations, db2 gave the algebra and SQL that read them, db3 removed the redundancy in them, and db5 moves on to transactions, concurrency and recovery. What sits between all of those is this lesson's one question: **the tables are designed and clean — but where are their bytes, and how does the DBMS reach one row among millions?**

High-yield list

① Heap: unordered, insert fast
② Sequential: sorted, search fast
③ Hashed: `h(k)=k mod N`
④ Indexed: extra space, very fast
⑤ Primary store = RAM, volatile
⑥ Bit → Byte → Block → File → DB
⑦ Primary index = on the search key
⑧ **Only ONE clustered index ★**
⑨ Dense = per record, sparse = per block
⑩ B+ tree: all keys in the leaves ★
⑪ Hash: O(1) for `=`, no range
⑫ Plan: SQL→Parser→Optimizer→Plan

Legend: **yellow** = key term · **red circle** = must-memorise · **purple** = memory trick · **red** = exam trap · **green** = worked example

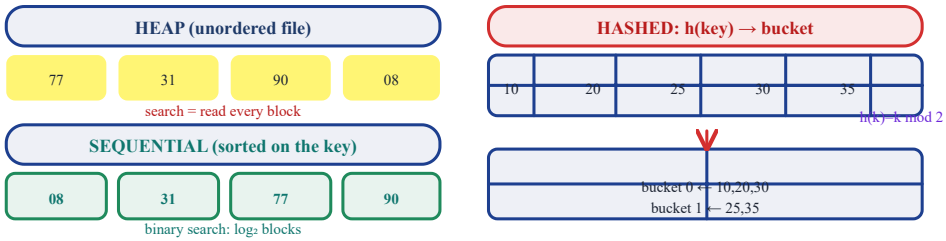
Contents

✓ File organization, four kinds … p. 2
✓ Storage types & the unit ladder … p. 3
✓ Indexes: the four types … p. 4
✓ Inside an index: dense, sparse … p. 5
✓ B+ trees and the leaf chain … p. 6
✓ Hash tables & collisions … p. 7
✓ Query optimization, stage by stage … p. 8
✓ Cost model & index design … p. 9
✓ Night-before sheet … p. 10
✓ Practice MCQs + written … p. 11

1 File Organization: How Records Actually Live on Disk

Definition: **File organization** is the way **records are stored physically** inside the data file — their order on disk and the path the DBMS follows to reach one. It is a *physical* decision: the same table can be stored four different ways, and the four ways have very different costs for insert, delete and search.

Organization	Description and characteristics
Heap (unordered)	Records sit in **no order at all**. Insertion is fast — just append. Search is slow, **O(n)**: every block must be read.
Sequential	Records are **sorted by a key field**. Search on that key is fast, but insertion is slow — every later record must be shifted.
Hashed	A hash function computes the storage location, giving **direct access**. Equality search is fast; **collisions are possible**.
Clustered / Indexed	Data is stored **with index pointers** beside it. Search is very fast; the index costs **extra space**.



the same five records stored three ways — the order on disk is the whole difference

Organization	Insert	Delete	Search	Space
Heap	**Fast**	Moderate	Slow	Minimal
Sequential	Slow	Slow	**Fast**	Minimal
Hashed	Moderate	Moderate	**Fast**	Moderate
Indexed	Moderate	Moderate	**Very Fast**	Extra

Memory trick: "**Order on disk buys search and sells insert.**" The heap keeps no order, so nothing is ever moved — that is why its insert is Fast. The sequential file keeps an order, so every new key may shove half the file along — that is why its insert is Slow. The four-column table *is* the trade, and it is **frequently tested** as a table.

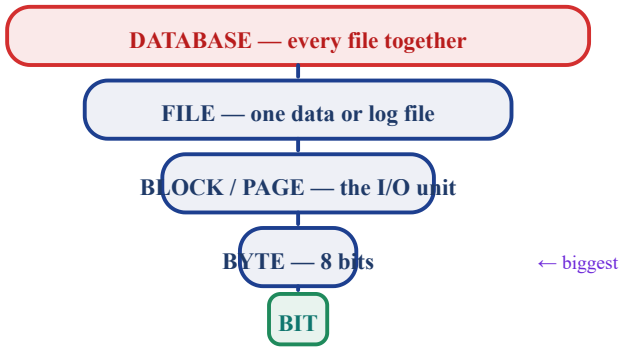
Exam trap ✍ three: **1.** "Sequential insert is slow" means slow **because records must be shifted**. **2.** Hashed search is Fast for **equality only** — there is no order to exploit for a range. **3.** "An indexed file costs no extra space" is **false**; the index is a second structure, which is why Indexed is the only row marked Extra.

Where the chapter lives: db1 designed the relations, db2 gave the algebra and SQL, db3 removed the redundancy. This lesson asks a different question: **where do the bytes sit, and how does the DBMS reach a row?**

2 Data Storage Types and the Storage-Unit Ladder

Where the bytes physically live is a different question from how they are arranged inside a file. The DBMS keeps working data in three **tiers**, and inside each tier it addresses data in five **units**.

Storage type	Description	Remember
Primary (RAM)	Fastest access — the CPU reads it directly. **Volatile:** data is lost when power fails.	buffer pool, temp tables
Secondary (HDD / SSD)	**Persistent** — survives power failure. Slower than RAM; this is where the data files, the log files and the indexes live.	the database itself
Tertiary (Tape / CD)	Very slow, sequential access only. Used for **archiving** and backups.	offline, cold copies



the ladder every I/O cost is counted in

The ladder, in words: **Bit → Byte → Block/Page → File → Database.** A byte is 8 bits; a **block (page)** is the fixed-size unit the DBMS actually reads from disk — typically 4 KB or 8 KB; a file is a named sequence of blocks; the database is all the files together. **Every disk I/O counts blocks, never bytes** — which is exactly why the blocking factor matters.

Blocking factor ✍️ the number you will compute: The **blocking factor** is how many records fit in one block: `bfr = ⌊ block size ÷ record size ⌋`. The file occupies `⌈ records ÷ bfr ⌉` blocks. **Unspanned** means no record is split across two blocks. Take the **floor** for the blocking factor and the **ceiling** for the block count.

Exam trap ✍️ four: **1.** "**Primary** storage" means RAM — not "primary key", not "the first level" — and RAM is volatile. **2.** An SSD is **secondary** storage: persistent, however fast it is. **3.** Tertiary storage is **archiving** and is essentially sequential — you cannot seek into a tape. **4.** Speed and volatility both fall as you go down: primary is fastest and forgetful, tertiary is slowest and permanent.

Memory trick: "**Primary is quick but forgetful; secondary is the database's own home; tertiary is the attic.**" And for the ladder: "**many bits make a byte, many bytes make a block, many blocks make a file, many files make a database.**" Every step is "many of the thing below", and the block is the only one the hardware moves in one operation — which is why a cost model counts pages, not characters.

3 Indexes: The Definition and the Four Types

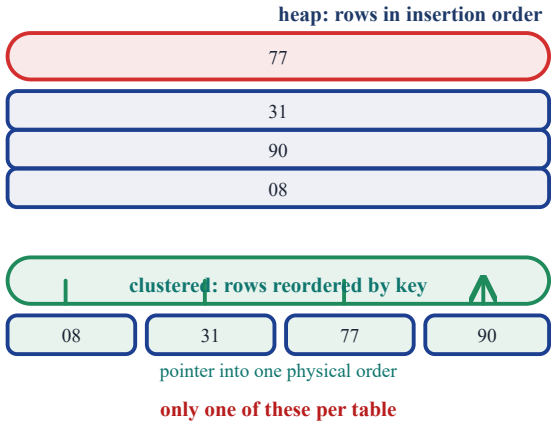
Definition: An **index** is a **data structure that speeds up retrieval** by keeping an extra sorted map from a search key to the location of the row. It does not hold the data — it holds the **path** to the data, so a search touches a tiny fraction of the blocks instead of all of them.

Index type	Description
Primary index	Built on the **primary (search) key**. Because that key is unique and never changes, the index **physically orders the data file** — the records sit in key order on disk.
Secondary index	Built on a **non-key column** (a foreign key, or any attribute you often filter on). It provides an **alternate path**; the data itself is not reordered, so the key may repeat.
Clustered index	The **data is physically ordered by the index key**. Reading a range in key order is then a single sequential sweep of the file.
Non-clustered index	A **logical pointer** to the data held in a separate structure; the rows stay where they are and the index points at them.

Exam trap ✍ the one to memorise: **At most ONE clustered index per table.** A clustered index fixes *the physical order in which rows are stored*, and a set of rows can be stored in only one order at a time — so a second clustered index would have nothing left to decide. A table may have **any number of non-clustered (secondary) indexes**, one per column you filter on. "Two", "one per column" and "any number" are all wrong for the clustered case.

Primary vs clustered — do not confuse: These two travel together but are not synonyms in a textbook question. **Primary index** is about *which key* (the search key, and the file is ordered by it). **Clustered index** is about *what is physically true* (the rows sit in that order). A **secondary** index never reorders the file, so a table carrying only secondary indexes is still a heap on disk.

What a non-clustered leaf entry holds: `<key value, row locator>`. The locator is the **primary key** of the row if one exists; if the primary key is a surrogate — an auto-generated id the user never sees — the locator is the **RID**, the record identifier, effectively its physical address. Only after the locator is followed is the table itself touched at all.

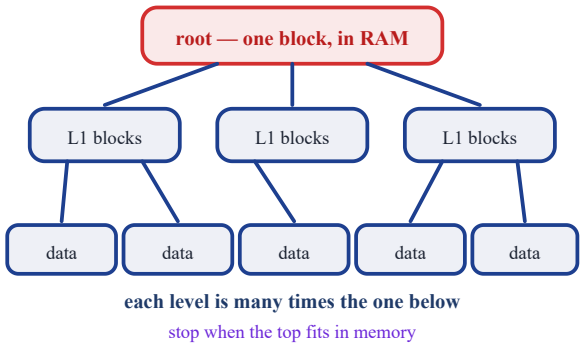


a clustered index moves the rows; a non-clustered one moves only the pointers

4 Inside an Index: Dense, Sparse, Multilevel — and What an Index Costs

Kind	One entry for…	Size	Search cost
Dense index	every **record**	large — grows with the file	small index, fastest lookup
Sparse index	every **data block** — pointing to the first key in that block	small — grows with blocks only	bigger index, but far fewer blocks to search

Multilevel index — an index on the index: A sparse index still has to be searched, and if it does not fit in memory, searching it means reading *its* blocks. So the DBMS builds a **second level on the index**, and a third, until the top level is one block that always stays in RAM. A **multilevel (B-tree / B+ tree) index** is exactly this: the top level is in memory and each further level shrinks by the fan-out factor. That is why a search over 100 million records still costs only a handful of disk reads.



multilevel: the index itself becomes a searchable tree

Exam trap ✍️ sparse needs a sorted file: A **sparse** index stores one entry per block, so its entry must say "this is the *first* key in that block" — a statement that is only meaningful if the blocks are in **key order**. That is why a sparse index is built over a **sorted (sequential)** file. A **dense** index has an entry per record, so it can be built over an **unordered heap** as well. The trade is pure: dense uses more space and fewer reads; sparse uses less space and more reads.

What every index costs you: **1.** Extra space — the index is a second copy of the keys. **2.** Every INSERT and DELETE must also update the index, so writes get slower. **3.** The optimizer must consider each one when it picks a plan. **4.** An unused index is pure cost. **Over-indexing** — an index on every column — is a real trap: it makes writes slow, wastes space and confuses the optimizer, all for no benefit.

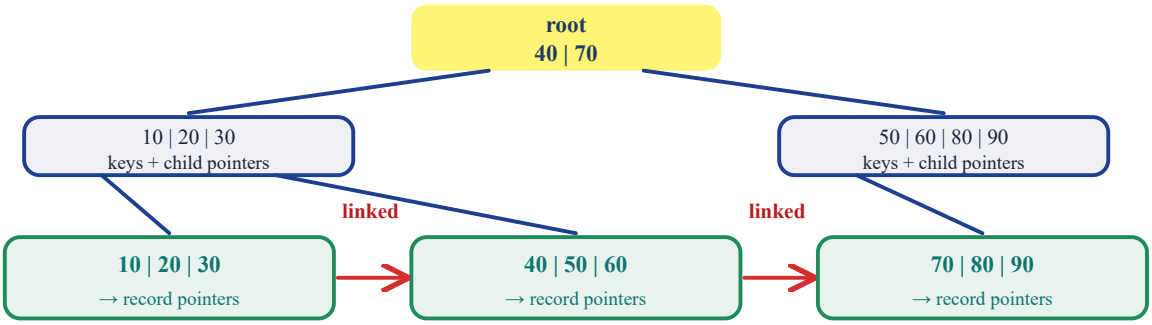
Memory trick: **"An index is a price tag, not a copy."** It carries the key and the locator, never the payload. And **"dense is short and fat, sparse is thin and tall"** — dense has many entries in few blocks; sparse has few entries in a taller tree, though a multilevel sparse index closes most of the gap anyway. Never index a column just because it exists: **index what you filter, join or sort on.**

Where the index physically lives: The index is **not** inside the data file: it is a separate set of blocks on the same disk, and the DBMS is free to keep the hottest levels in the buffer pool in RAM. That is the whole reason a multilevel index is fast — the top of the tree never touches the disk. A search therefore costs a few RAM reads plus, at worst, one or two disk reads, whatever the size of the table.

5 B+ Trees: the Workhorse Index, and the Leaf Chain That Makes Ranges

Fast

Definition: A **B+ tree** is the most common indexing structure in databases: a **balanced tree** in which **every leaf sits at the same level**. Internal nodes hold only **keys + child pointers** and exist purely to guide the search; leaf nodes hold **keys + record pointers**. Because the tree is balanced and its **fan-out is high** — hundreds of keys in one node — the tree is only a few levels deep, so search, insert and delete are all **O(log n)** with a tiny constant.



internal nodes repeat keys only — they never touch data

every key lives in the leaves, and the leaves are chained in key order

one level for all leaves = balanced · high fan-out = very few levels

a correct B+ tree: all leaves at one level, all keys in the leaves, leaves linked left to right

Why it wins on range queries ✍️ the standard answer: In a B+ tree **every key, with its record pointer, is stored in the leaves**, and **the leaves are chained together in key order**. So for `WHERE salary BETWEEN 20000 AND 30000` the index first finds the **first leaf for 20000** in **O(log n)** — an ordinary tree descent — and then simply **follows the leaf chain**, reading leaf after leaf, until it passes 30000. Internal nodes do no work beyond that first descent.
That is why a range query on a B+ tree index costs O(log n) + k, where k is the number of matching entries.

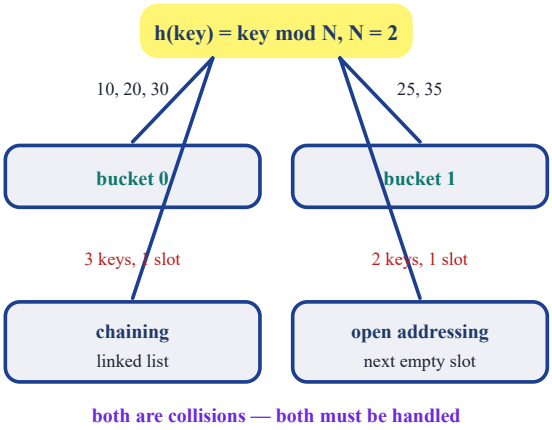
Exam trap ✍️ B-tree vs B+ tree — three wrong answers to kill: "It has **no internal nodes**" is false — internal nodes exist and guide the descent. "It stores **all records in the root**" is false — the root holds only keys and child pointers. "It is **always a binary tree**" is false — its fan-out is high, so a node holds hundreds of keys and thousands of children. The only correct statement is the one about **leaves holding every key and being linked in key order**. In a plain **B-tree** records sit in *all* nodes, so a range scan must jump around; in a **B+ tree** only the leaves carry records, so a range scan is one continuous walk.

Point	B-tree	B+ tree
Where the data sits	in **every** node, internal included	**only in the leaves**
Range query	must jump between levels	**one walk down the leaf chain**
Used in practice	file systems, mostly	**database indexes, almost always**

6 Hash Tables: O(1) Equality — and Why Ranges Kill Them

Definition: A **hash table** computes the storage location *directly*: a **hash function** maps the key onto a bucket, so no comparison and no traversal is needed. `h(key) = key mod N`, where **N is the number of buckets**.

Worked example ✍ with N = 2 stated: Here **N = 2**, so there are exactly two buckets and `h(key) = key mod 2`: `10 → 0` · `20 → 0` · `25 → 1` · `30 → 0` · `35 → 1`
So **bucket 0 = [10, 20, 30]** and **bucket 1 = [25, 35]**. The keys 10, 20 and 30 all hash to 0 — that is a **collision**: different keys competing for one slot. Every one of these values is consistent **only** with N = 2; with N = 10 the answers would all differ, so **always state N before computing.**



two buckets only — a key either collides or it does not

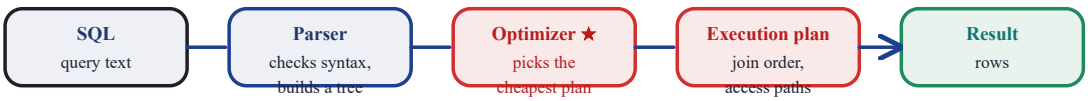
Collision handling	How it works
Chaining	Each bucket holds a **linked list**; a colliding record is appended to that bucket's list and a search walks the short list. Bucket 0 above is the chain 10 → 20 → 30.
Open addressing	No list. If the computed slot is taken, **probe forward to the next empty slot** and put the record there. When the table fills up it must be rehashed larger.

Hash vs B+ tree: **Hash:** equality lookup **O(1) average** — nothing better exists. But a hash function destroys order, so **range queries, ORDER BY and MIN/MAX are hopeless**. **B+ tree:** slightly slower for a single equality hit, but it keeps the keys in order, so ranges, sorting and "next record" are all natural. Choose a hash for **exact-match on a huge, stable key**; choose a B+ tree the moment the query needs a range or an order.

Exam trap ✍ three: **1.** A hash index answers "salary = 25000" in one step but **cannot** answer "salary BETWEEN 20000 AND 30000" without reading everything — so the high-yield pairing is **B+ tree for BETWEEN, hash for =. 2.** "A hash table has no collisions" is false the moment two keys hash alike. **3.** N should be larger than the number of distinct keys if collisions are to be rare, but a huge N wastes space — real implementations pick N as a fraction of the expected record count and rehash as the file grows.

7 Query Optimization: From SQL Text to an Execution Plan

Definition: **Query optimization** is the process of **choosing the most efficient execution plan** for a SQL query. SQL says *what* you want; several plans may return the right rows, and the optimizer picks the one that costs least to run.



the same five stages every DBMS follows

Technique	Description
Use indexes	Put an index on every column used in **WHERE, JOIN and ORDER BY** — those are the only three places an index can ever be used.
Avoid SELECT *	Select only the columns needed. Unused columns cost extra **I/O** *and* stop the query from being answered by a covering index alone.
Efficient joins	**Hash join** for large unsorted inputs · **Merge join** when both inputs are already sorted · **Nested loop** when one side is tiny.
Avoid subqueries	Rewrite an `IN (SELECT …)` as a **JOIN** or an `EXISTS`; the optimizer can then choose a join strategy instead of running the inner query repeatedly.
Query hints	DB-specific directives written into the query to steer the optimizer when its statistics-based guess is wrong.
Partitioning	Split one large table into partitions so a query touches only the partition it needs.
Caching	**Materialized views** store the result of a query so it can be read directly; the query cache reuses a plan and its result.

Exam trap ✍ two: **1.** The optimizer does **not** change what the query returns — it changes **how** the answer is produced. Two different plans returning the same rows are both correct; one is cheaper. **2.** "The optimizer chose a slow query" usually means the **statistics are stale** or a **useful index is missing** — not that the optimizer is broken.

Memory trick: "**Index what you filter, join or sort on — those three words are the whole list.**" An index on any other column is never used by that query, however beautiful it looks. And "**avoid SELECT ***" is not tidiness: it is wasted I/O plus the loss of a covering-index-only plan.

Reading a plan ✍: Every real DBMS will show you the plan it chose — `EXPLAIN` in most of them — and the plan is not a secret. It lists, in order, the tables to scan or index, the **join order**, and the **join algorithm** used at each step. For a written question, "the plan scans Employee first with a full table scan, then indexes Department by its primary key" is a complete and correct answer; naming the physical structure is what earns the marks, not naming the SQL again.

8 Cost-Based vs Heuristic Optimization — and How to Make the Plan Good

The cost model ✍ three factors: For every candidate plan the optimizer estimates:
1. I/O cost — the number of **disk page reads and writes**. This dominates, because a disk read is millions of times slower than a CPU operation.
2. CPU cost — the number of comparisons, tuple operations and expressions evaluated.
3. Communication cost — the data shipped between sites; it counts only in a **distributed** database.

Cost-based (statistics) optimizer: Uses **statistics** — rows per table, distinct values, data distribution — to estimate the I/O and CPU cost of every candidate plan and pick the **least-cost** one. Powerful, but wholly dependent on the statistics being current.

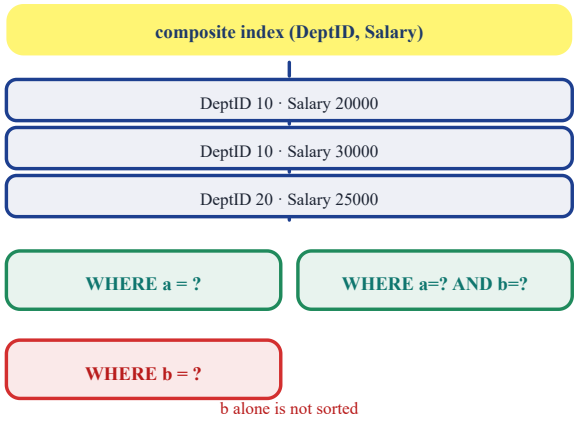
Heuristic (rule-based) optimizer: Applies **fixed transformation rules** regardless of the data: perform **SELECT early** to reduce the number of rows as soon as possible; perform **projections early** to reduce the number of columns as soon as possible; perform the **most restrictive join first**. It cannot measure, only transform — modern systems run cost-based optimization with these heuristic rules layered on top.

Index selectivity and column order: **Selectivity** is the fraction of rows a predicate keeps. A **highly selective** condition — few rows surviving — is what makes an index lookup worthwhile; a predicate that keeps most rows is not. In a **composite index** put the **more selective column first**, because that is the one that narrows the search.

Exam trap ✍️ the leftmost-prefix rule: A composite index `(a, b)` can serve `WHERE a = ?` **and** `WHERE a = ? AND b = ?`, but it cannot serve `WHERE b = ?` alone — the index is not sorted by `b` on its own, so every value of `b` is scattered across the whole tree. This is the **leftmost-prefix rule**, and it is **frequently tested**. The same rule explains why `ORDER BY a, b` is free with that index while `ORDER BY b` is not.

Covering index — the strongest reason to drop SELECT *:

If **every column the query needs is already in the index**, the optimizer answers the query **without touching the table at all**; this is an index-only scan. Adding a column the query does not need — the habit behind `SELECT *` — destroys the covering index and forces a trip to the data rows, in addition to the extra I/O of reading those columns. A covering index therefore trades a little write time for a large read saving.



leftmost-prefix: a, then a+b — never b on its own

Night-Before-the-Exam Sheet

Fact sheet:

Heap: unordered, insert Fast	Heap search O(n), space Minimal
Sequential: search Fast	Sequential insert Slow (shifting)
Hashed: `h(k) = k mod N`	Indexed: Extra space, very fast
Primary store = RAM, volatile	Secondary = disk, persistent
Tertiary = tape, archiving	Bit → Byte → Block → File → DB
`bfr = ⌊size ÷ recsize⌋`	blocks = ⌈records ÷ bfr⌉
Primary index: search key	Secondary index: non-key column
Clustered: data physically ordered	Non-clustered: pointer only
Only ONE clustered index	Dense = entry per record
Sparse = entry per block	Multilevel = index on the index
B+ tree: all keys in leaves	B+ leaves chained in key order
B+ search / insert = O(log n)	B+ range = O(log n) + k
Hash equality O(1) average	Hash: **no range query**
SQL → Parser → Optimizer	→ Execution plan → Result
Cost = I/O + CPU + comms	Hash join: large · Merge: sorted
Leftmost-prefix rule	Covering index: no table touch

Top 12 one-liners for MCQs

1. Heap = no order ⇒ fast insert, O(n) search
2. Sequential = sorted ⇒ fast search, slow write
3. Primary storage (RAM) is volatile; disk is not
4. Blocking factor floors, block count ceilings
5. A table has at most **one** clustered index
6. Sparse index ⇒ one per block, sorted file
7. In a B+ tree every key sits in the **leaves**
8. B+ leaves are **chained** — that is why BETWEEN is fast
9. A B+ tree is **not** a binary tree
10. Hash is O(1) for `=`, useless for BETWEEN
11. Optimizer cost = I/O + CPU + communication
12. Composite index serves `a` and `a,b`, never `b` alone

Top traps: a **hash index** serving a range · several **clustered indexes** · keys **only in internal nodes** of a B+ tree · calling the B+ tree a binary tree · `h(key)=key mod N` **without giving N** · forgetting the **ceiling** · indexing an unused column.

All tricks together: Order on disk buys search and sells insert · primary is quick but forgetful, disk is home, tape is the attic · an index is a price tag · dense is short and fat, sparse thin and tall · linked leaves are the range answer · hash for `=`, tree for BETWEEN.



three structures, one plan

✓ Practice MCQs (exam pattern, 10 marks)

1. A **heap** (unordered) file is chosen because:
(a) Search on any attribute is fast (b) Insertion is fast, no order to keep
(c) It needs no extra space (d) It supports range queries

2. Which storage type is **volatile**?
(a) Secondary (HDD/SSD) (b) Tertiary (tape)
(c) Primary (RAM) (d) None — all are persistent

3. At most how many **clustered** indexes can one table have?
(a) Two (b) One per column
(c) Only one (d) Any number

4. A **sparse** primary index has one entry for:
(a) Every record (b) Every data block
(c) Every attribute (d) Every secondary key

5. Why is a B+ tree efficient for `salary BETWEEN 20000 AND 30000`?
(a) All keys are in the leaves, which are linked in key order (b) It has no internal nodes
(c) It keeps all records in the root (d) It is always a binary tree

6. 30,000 records of 100 bytes, unspanned in 1024-byte blocks, sorted on the key. A **sparse** index has one 15-byte entry per data block. How many index blocks?
(a) 44 (b) 45
(c) 442 (d) 30

7. With `h(key) = key mod N` and **N = 2**, keys 10, 20, 25 hash to:
(a) 0, 0, 1 (b) 1, 1, 0
(c) 0, 1, 0 (d) 2, 2, 1

8. A hash index answers which query best?
(a) `salary BETWEEN 20000 AND 30000` (b) `ORDER BY salary`
(c) `WHERE empid = 4471` (d) `SELECT MIN(salary)`

9. A composite index on `(a, b)` can serve:
(a) `WHERE b = ?` alone (b) `WHERE a=?` and `WHERE a=? AND b=?`
(c) `ORDER BY b` alone (d) Nothing — composites are unusable

10. A cost-based optimizer estimates:
(a) I/O, CPU and communication cost (b) Only the rows returned
(c) Table size and column count (d) The alphabetical order of tables

Answer key → 1-b · 2-c · 3-c · 4-b · 5-a · 6-b · 7-a · 8-c · 9-b · 10-a

Written question: Define file organization and give the Insert / Delete / Search / Space profile of all four types. A data file has **30,000 records of 100 bytes**, unspanned in **1024-byte blocks**, sorted on the key. A **sparse** primary index has one **15-byte** entry per data block. How many index blocks? Show every step, compare with a dense index, and say why a B+ tree beats a hash on `BETWEEN`. 10 marks

1. **Four organizations (2)** — **Heap**: no order, insert Fast, delete Moderate, search Slow O(n), space Minimal. **Sequential**: sorted, insert Slow (records shift), delete Slow, search Fast, Minimal. **Hashed**: Moderate except search Fast. **Indexed**: search Very Fast, space Extra. 2
2. **Data blocking factor (2)** — unspanned means no record crosses a block: `bfr = ⌊1024 ÷ 100⌋ = 10`, so the file is `30000 ÷ 10 = 3000` blocks. 2
3. **Index blocks (2)** — sparse ⇒ **3000** entries; `bfr = ⌊1024 ÷ 15⌋ = 68` per block; blocks = `⌈3000 ÷ 68⌉ = ⌈44.1⌉ = 45` — the ceiling is needed because the last block is partly full. A **dense** index ⇒ `⌈30000 ÷ 68⌉ = 442` blocks. 2
4. **B+ tree vs hash for the range (3)** — every key with its record pointer sits in the **leaves**, and the leaves are **chained in key order**: descend to the first leaf holding 20000 in **O(log n)**, then follow the chain until the entries pass 30000 — cost O(log n) + k. Internal nodes hold only keys and pointers; the fan-out is high, so it is not a binary tree. A hash index jumps to `h(salary)`: O(1) for `=`, but keys are stored in **no order**, so BETWEEN needs every bucket read — **tree for ranges, hash for equality**. 3
5. **Add the rules (1)** — at most **one clustered index** per table; a sparse index needs a **sorted** file; a composite index follows the **leftmost-prefix rule**. 1

Before the exam ✓

- ✓ Can name the four file organizations
- ✓ Can reproduce the four-column table
- ✓ Can compute a blocking factor
- ✓ Can draw a B+ tree with its leaf chain
- ✓ Can separate dense from sparse
- ✓ Can list the five query stages